ECC Focus Queue — Backend Interface Guide for Agents

Audience: AI agents (Codex, Claude, Kimi), the orchestrator, the router agent, and any external program that needs to interact with ECC Focus Queue without the UI.

Purpose: This is the complete command-level contract for the app's backend interface: every CLI command, every data file, every environment variable, and — critically — the *correct* way to register tasks, agents, and sessions so the app stays consistent. It ends with a failure analysis and a list of missing commands and hardening proposals.

Sources: `Sources/FocusQueueSession/main.swift`, `Sources/FocusQueueCore/*`, `Sources/ECCFocusQueue/*`, `Scripts/focus_queue_*.py`, `Config/*.json`, `Docs/AGENT_CONTRACT.md`, `Docs/ACTION_BOUNDARIES.md`, `Docs/REMOTE_SESSION_RECALL.md`, `Docs/DATA_BACKFILL_AUDIT.md`.

---

1. Architecture at a glance

There is **no daemon API, no URL scheme, no XPC, no AppleScript**. The entire backend interface is:

1. **One Swift CLI binary** — `focus-queue-session` (installed at `~/.local/bin/focus-queue-session`, also inside the app bundle at `Contents/MacOS/focus-queue-session`). This is the authoritative mutation path. All output is pretty-printed JSON on stdout; errors go to stderr with exit 1.
2. **Python harness CLIs** — `focus-queue-codex`, `focus-queue-claude`, `focus-queue-kimi`, `focus-queue-report`, plus connector CLIs (`focus-mail`, `focus-discord`, …).
3. **A shared file tree** under the support directory, guarded by `flock` lock files.
4. **Two distributed notifications** the app listens for.

1.1 The support directory

Everything roots at:

```
~/Library/Application Support/ECC Focus Queue/
```

overridable with `FOCUS_QUEUE_SUPPORT_DIR` (used by tests to isolate state). Key layout:

Path	Contents
`config.json`	`AppConfiguration` — projects, providers, verifier, daily report
`state.json`	`LocalState` — UI state **and** `agentSessions[]` (session registry)
`tasks/<TICKET>/context.json`	`TaskContext` — subtasks, workspaces, tmux refs, per-task state
`tasks/<TICKET>/plan.md`	Task plan (Markdown)
`tasks/<TICKET>/agent-log.md`	Append-only agent log

Path	Contents
`tasks/<TICKET>/archive/snapshot-*`	Context snapshots (max 100)
`agent-registrations/<sha256>.json`	Session registration inbox
`external-agent-sessions/<sha256>.json`	Hook registration receipts
`tracking/placements/<launch-uuid>.json`	Placement capsules
`tracking/launches/` , `tracking/resolutions/`	Intake receipts, launch→session resolution
`agent-routes/`	Router route intents and router slots
`queue-order/order[-<b64 project>].json`	Queue ordering per project
`logs/agent-startups/<TASK>/<subtask>.json`	Startup receipts
`library/`	Durable cross-device history
`ingestion/`	Daily-report state, reports, launch requests
`<dashboardPath>` (outside support dir)	Per-project `dashboard.json` — the ticket list

The live dashboard is at `config.json`'s `dashboardPath` (currently `/Users/hira/Documents/ECC/ecc-dashboard/data/dashboard.json`), **not** inside the support dir.

1.2 Locking discipline

All writers — Swift and Python — take advisory `flock` locks:

- `.<filename>.focus-queue.lock` next to any locked file (dashboards, config)
- `tasks/<TICKET>/.lock` for per-task context mutations
- `queue-order/order.lock` , `agent-registrations/.inbox.lock`

Any program that writes these files without taking the same locks risks torn reads and lost updates against the app. Writes must also be atomic (`write temp → rename`), mode 0600, and **never through a symlink** — symlinks are treated as hostile and rejected almost everywhere.

1.3 How the app notices changes

- `com.haelyra.ecc-focus-queue.agent-registration` — posted by every `focus-queue-session` mutation; the app imports pending registrations and refreshes.
- `com.haelyra.focus-queue.project-catalog-change` — posted by `projects create` ; the app reloads `config.json` .

Everything else is **polled**: dashboards and task contexts are re-read when mtime/size/inode changes (2 s tick while the panel is visible, 30 s hidden, forced re-read every 60 s). **Direct edits to `config.json` are ignored until restart** (or a project-catalog notification), and **direct edits to `state.json` are worse than ignored** — the app holds a cached copy in memory and will overwrite your edits on its next save.

---

2. The environment contract

Agents receive and consume identity through environment variables. These are the backbone of registration — if they are missing or stale, registration silently degrades.

Variable	Set by	Meaning
`FOCUS_QUEUE_SUPPORT_DIR`	launcher/tests	Support dir override
`FOCUS_QUEUE_TASK_ID`	managed launch	Ticket this agent belongs to
`FOCUS_QUEUE_SUBTASK_ID`	managed launch	Subtask (agent slot) ID
`FOCUS_QUEUE_LAUNCH_ID`	entry wrapper	Launch UUID → placement capsule path
`FOCUS_QUEUE_AGENT_CONTEXT`	entry wrapper	Direct path to the placement capsule JSON
`FOCUS_QUEUE_TMUX_NAME`	managed tmux	Owning tmux session name
`FOCUS_QUEUE_PROJECT_ID`	queue commands	Scope guard for `queue` mutations
`FOCUS_QUEUE_PROVIDER`	entry wrapper	`codex` / `claude` / `kimi`
`FOCUS_QUEUE_ROUTER` / `_SOURCE` / `_CAPABILITY`	router launch	One-use router capability auth
`CODEX_THREAD_ID` / `CODEX_SESSION_ID`	Codex runtime	Provider session UUID
`CLAUDE_SESSION_ID`	Claude runtime	Provider session UUID
`WEZTERM_PANE` , `WEZTERM_UNIX_SOCKET`	WezTerm	Terminal pane identity
`TMUX` , `TMUX_PANE`	tmux	tmux identity

Known gap: there is no `KIMI_SESSION_ID` -style variable in the session-binding code. Kimi panes cannot be identified from env alone; Kimi must register through hooks (`register-event`). The `attach` usage text claims Kimi support it cannot deliver from env (`main.swift:17` , `TaskContext.swift:14-25`).

---

3. `focus-queue-session` — command reference

Canonical usage string lives at `Sources/FocusQueueSession/main.swift:16` . Every command below emits JSON.

3.1 Session registration (use these, in this order)

`focus-queue-session attach <TASK-ID>`
The mandatory first command for any agent that knows its ticket. Reads the provider session UUID from env (`CODEX_THREAD_ID` / `CODEX_SESSION_ID` / `CLAUDE_SESSION_ID`), registers the current WezTerm/tmux pane into `state.json.agentSessions` , sets the terminal label (`<Task name>` or `<Task name> · tmux`), and posts the registration notification.

Output keys: `status, registrationID, provider, sessionID, taskID, subtaskID, agentID, parentSessionID, placementPath` .

Pitfalls:

- Fails with `missingSessionIdentity` if the env session ID is absent **or not a UUID** (Kimi ULIDs fail here — use `register-event` instead).
- Correctly ignores a stale inherited `FOCUS_QUEUE_SUBTASK_ID` — re-attach after switching tickets, every time.
- **Does not validate that the ticket exists on a dashboard.** Attaching to a typo'd ticket registers a session for a task that doesn't exist.

`focus-queue-session register-event --provider codex|claude|kimi [--task <TASK-ID>]`
The hook-ingestion path; reads **one JSON `ExternalAgentEvent` from stdin** (≤1 MB, oversized → exit 1). This is how provider hooks (SessionStart, UserPromptSubmit, SubagentStart/Stop, ProcessDiscovery) register identity. Payload (snake_case):

```
{
  "session_id": "<uuid; required>",
  "cwd": "/absolute/path; required",
  "hook_event_name": "SessionStart",
  "prompt": "optional first prompt",
  "agent_id": "optional",
  "parent_session_id": "optional",
  "focus_queue_discovered_subagent": false
}
```

A non-UUID `session_id` is accepted but degrades the record to a non-provider identity. Malformed JSON → exit 1.

`focus-queue-session identify <TASK-ID> <SUBTASK-ID> <title> <description>`
Sets/replaces the agent's display title and assignment description. Hook-independent fallback for naming an `Agent N` slot.

3.2 Placement

`focus-queue-session placement [--prompt <text>] [--task <ID>] [--launch <UUID>] [--cwd <path>] [--no-create]`
Runs the deterministic placement resolver and writes a capsule to `tracking/placements/<launch-uuid>.json`. The capsule (`AgentPlacementCapsule`) is the agent's source of truth: `projectID, taskPrefix, taskID?, agentID?, provider, workspacePath, branch?, tmuxName?, tmuxPaneID?, projectPlanPath, taskPlanPath?, taskLogPath?, status (placed|project-inbox|blocked-conflict)`.

Rule: read your capsule (`$FOCUS_QUEUE_AGENT_CONTEXT`) before any source edit. Stop on `blocked-conflict`.

3.3 Routing (orchestrator/router path)

`focus-queue-session route inspect --prompt <text> [--cwd <path>] [--provider codex|claude|kimi]`
Read-only. Saves the full objective, returns a compact routing packet: inferred project, open-task candidates with plan/context/log paths, and exact `route start` command templates. Creates nothing.

`focus-queue-session route start <ROUTE-UUID> (--task <TASK-ID> | --project <PROJECT-ID> --task-title <T>) --title <T> --description <D> [--provider p] [--model m] [--effort low|medium|high|xhigh|max] [--repository PATH] [--workspace existing|isolated-clean|isolated-snapshot]`
The full managed-launch orchestrator: claims the route (idempotent — replay returns `replayed: true` instead of a second process), optionally creates the dashboard ticket, provisions the worktree, writes the subtask and a startup receipt (`logs/agent-startups/<TASK>/<subtask>.json`), launches the provider in the task tmux session, and waits ~10 s for the provider session UUID.

Result: `status, routeID, taskID, subtaskID, providerSessionID, startupReceiptPath, routerAction (exit|wait-for-registration), replayed, startedAt` .

Pitfall: **unknown `--flags` are silently swallowed** (`main.swift:370-377`) — `--reposatory /path` is accepted and ignored. Typos in flag names produce successful-looking JSON with the option dropped.

3.4 Task mutations

`task rename <TASK-ID> <title...>` — revision-checked rename.
`task priority <TASK-ID> critical|high|medium|low` — validated enum.
`task status <TASK-ID> <status>` — ⚠ **accepts any string.** No enum validation. A typo (`complted`) writes successfully and the ticket just stays "open" forever. See §8.

All three go through `JSONDashboardRepository` with optimistic-concurrency revision checks, duplicate-ID rejection, and type-safe re-encoding — this is what hand-editing `dashboard.json` bypasses.

There is no `task create` , no `task show` , no generic `task set <field>` . See §8 — this is the single biggest gap.

3.5 Queue ordering

`queue list` · `queue reset` · `queue move <TASK-ID> top|bottom|before <ID>|after <ID>` · `queue reorder <TASK-ID>...`
Scoped by `FOCUS_QUEUE_PROJECT_ID` . Writes `queue-order/order-<b64(project)>.json` under `order.lock` . `queue reset` restores priority order. Emits `{"order": [...]}` .

3.6 Subtask (agent) lifecycle

These manage the per-agent slots inside `tasks/<TICKET>/context.json` :

Command	Purpose
`add <TASK> <codex|claude|kimi> <title> <description>`	Create a queued subtask (title ≤200, description ≤2000 chars)
`rename <TASK> <SUBTASK> <title>`	Rename a subtask
`identify <TASK> <SUBTASK> <title> <description>`	Set title + assignment description
`progress <TASK> <SUBTASK> <queued|working|idle|blocked> <summary>`	Heartbeat/status update (validated enum)
`complete <TASK> <SUBTASK> <summary>`	Mark completed with result summary

Command	Purpose
`attention <TASK> <AGENT> <approval|question|interrupted|incomplete|failed> <summary>`	Flag the agent as needing the user
`result <TASK> <AGENT> <summary>`	Record a result without completing
`archive-agent <TASK> <SUBTASK>`	Archive a finished agent slot
`associate <TASK> <AGENT> <SUBTASK>`	Link a registered session to a subtask
`log <TASK> <AGENT> <entry>`	Append-only entry to `agent-log.md` (task-locked, concurrency-safe)
`context <TASK>`	Dump the full `TaskContext` JSON
`plan <TASK>`	Print `plan.md`
`workspace <TASK> <SUBTASK> <abs-path>`	Record the agent's workspace (immutable once set — first write wins)
`review-push <TASK> <SUBTASK>`	Gated push review; exit 1 if rejected
`agents clear --all`	⚠ Wipes **all** agent sessions/subtasks/registrations. No confirmation, processes keep running untracked.

3.7 Runtime diagnostics & repair

Command	Purpose
`runtime locate <provider> <session-id>`	Is this provider session alive? → `status (running|stopped|unknown), processID, tmux identity`
`runtime repair <TASK> <SUBTASK>`	Repair a broken runtime record
`diagnose <TASK>`	Read-only WezTerm/socket inspection
`reconcile <TASK> [--no-cleanup]`	Repair task documents, write a receipt
`architecture <TASK> [repository-path]`	Generate/update architecture docs

3.8 Projects

`projects list` · `projects recover` · `projects create --id <id> --name <name> --workspace <abs-path> --from <project-id> --task <TASK-ID> [--task ...]`
`create` is transactional and posts the project-catalog-change notification so the running app reloads `config.json`. This is the *only* safe way to add a project while the app runs.

3.9 Library (durable history)

`library migrate | status | task <TASK-ID> | document <TASK-ID> <file.md> [guide|architecture|reference] | export <dest> | restore <src>` — append-mostly cross-device history under `library/`.

3.10 tmux

`tmux list | panes <name> | save | restore | persistence | arrange <name> | manual <name> | reflow <name> [--server s] | start <name> [--cwd dir] [-- cmd...] | start-agent <TASK> <AGENT> [--cwd dir] -- cmd... | attach <name> [--pane %N] | attach-reference <json> [--task T --agent A] | reference <name> <%pane> [--task T --agent A] --server s | ssh-command <user@host> <name>`

Managed agent sessions are named `fq-<subtaskID>` on the dedicated `ecc-focus-queue` tmux server; task-shared sessions are `fq-task-<task-id>`; unmanaged inbox sessions are `fq-codex-*` / `fq-claude-*`. `attach*` ends in `execv` and requires a TTY.

3.11 Misc

`intake-changed` — emits `{"status":"notified"}` and posts the change notification.

---

4. Provider harness CLIs

Installed at `~/.local/bin/focus-queue-codex|claude|kimi`. All three share one contract: global `--runtime <dir>`, subcommands `chat | run | reset`, timeout validated to 10–900 s (exit 2 on invalid), failures exit 1 with `<Provider> Queue harness failed: ...`.

`chat --session <name> --workspace <dir> --prompt-file <path> [--timeout 120] [--model M] [--effort E] [--context-dir D] [--writable-dir D]*`
Persistent per-session conversation. Session map at `<runtime>/sessions.json` under `sessions.lock`; stale sessions auto-retry with a fresh session. Default runtimes: `agent-claude/`, `agent/`, `agent-kimi/` under the support dir.

`run --workspace <dir> --prompt-file <path> [--sandbox read-only|workspace-write] [--output-schema <json>] [--usage-file <path>] [--disable-tools] [--model M] [--effort E] [--label L]`
One-shot headless run. With `--output-schema`, the reply is validated against the schema with one correction retry; the validated JSON is printed. `--usage-file` receives token counts. Every run writes prompt-free `<runtime>/latest-run.json` diagnostics.

`reset --session <name>` — drop a saved session id.

Harness-specific sharp edges:

- **Kimi:** `--effort` is parsed and silently dropped; `--disable-tools` cannot actually disable Kimi tools (prompt contract only).
- **Claude:** `--sandbox read-only` without `--disable-tools` still runs with tools enabled (`--permission-mode bypassPermissions`). "read-only" is only the sandbox flag.

- **`--label`** is parsed but never used in both Codex and Claude harnesses — silent no-op.
- Several scripts hardcode `/Users/hira/...` defaults — not portable.

5. Reporting & connector CLIs

`focus-queue-report` (`Scripts/focus_queue_morning.py`) — the ingestion pipeline:

```
focus-queue-report {collect|dry-run|run|mail-snapshot}
  [--config PATH] [--runtime PATH] [--platform discord|telegram|email|github]*
  [--trigger scheduled|manual|chat] [--comprehensive] [--recheck]
  [--agent-provider codex|claude|kimi] [--agent-model M] [--agent-effort E]
```

- `collect` — gather, print, no writes. `dry-run` — classify and count proposals, no writes.
- `run` — validate proposals against `Config/task-proposals.schema.json` and apply them to the dashboard under the shared lock. Exit 0 ok, 2 if any source failed, 1 on error.
- This is currently **the only validated task-creation path** — the schema enforces `title, projectID, category, priority, summary, next, doneWhen, sourceKeys, sourceLinks` on every new task.

Connectors: `focus-mail` (`authorize|status|search|read|open`), `focus-discord` (`status|channels|search|read`), `focus_queue_gmail.py` , `focus_queue_connectors.py` (readiness report), `focus_queue_data_audit.py` (read-only structural inventory of `tasks/<TICKET>/` — checks exactly the fields that are often missing: `ticketID, title, summary, updatedAt, worktreePath, planPath, subtasks, successCriteria`).

`focus-queue-map` — `snapshot <TASK>` , `diagram <TASK> [--view typical|context|agent]` , `scope <TASK> <SUBTASK> ...` , `resources --import` , `activity --import` . Task node/relation maps and Mermaid diagrams.

---

6. Data schemas (what "correct" looks like)

6.1 Dashboard ticket (`dashboard.json` → `tickets[]`)

```
{
  "id": "FOC-123",                    // required; unique across ALL project dashboards (case-
insensitive)
  "projectID": "focus-queue",
  "title": "...",                     // defaults to id if missing — always set it
  "category": "Bug",                  // Inbox|Bug|Feature|Maintenance|Community|Security
  "priority": "high",                 // critical|high|medium|low (default medium)
  "status": "backlog",                // default backlog; "resolved"/"cancelled" (case-insensitive)
= closed
  "summary": "What/why/how...",       // default "" — see §7.1
  "next": "next concrete action",
  "doneWhen": "verifiable completion condition",
  "updatedAt": "ISO-8601",
  "worktree": "...", "completion": {...}, "resolutionPrs": [], "prs": [], "links": [], "steps": []
}
```

Decode is lenient on missing fields but **strict on types**: `"prs": "x"` or `"steps": {}` fails decoding of the *entire dashboard* and the queue view errors out. A **duplicate ticket ID** in any dashboard fails the whole load. Unknown vocabulary strings (`status: "weird"`) pass decode and silently sort last.

6.2 Task context (`tasks/<TICKET>/context.json`)

`TaskContext` (`schemaVersion: 2`): `ticketID, title, summary, successCriteria, planPath, worktreePath?, repositoryPath?, branch?, baseBranch?, managedWorktree, subtasks[], projectID?, projectName?, updatedAt` .

Each `TaskSubtask` : `id (uuid), title, assignmentDescription, provider, sessionID?, status (queued|working|idle|blocked|completed), progressSummary, workspace?{path, repositoryPath, branch?, baseBranch?, baseCommit?, managed, ...}, tmux?, runtimeMode? (tmux|terminal), initialPrompt?, attention?, resultSummary?, launchConfiguration?{routeID?, model?, effort?}, ...` .

Closed vocabularies throw on unknown values: a hand-edited `status: "done"` or `provider: "gpt"` in `context.json` makes the *whole context file fail to load*. Missing fields are repaired with defaults on read — which is exactly why hand-written tasks "sort of work" but lack crucial pieces.

6.3 Session registry (`state.json` → `agentSessions[]`)

`TaskAgentSession{id, ticketID, provider, sessionID, tabTitle, taskTitle?, workingDirectory, windowID?, tabID?, paneID?, subtaskID?, tmux?}` . Upserted by (provider, sessionID). Written only by `attach` / `register-event` — **never edit `state.json` by hand** (the app's cached copy will overwrite you).

6.4 Task proposal schema (ingestion path)

`Config/task-proposals.schema.json` : `newTasks[]` requires `title, projectID, category, priority, summary, next, doneWhen, sourceKeys[1-20], sourceLinks[1-10]` . ⚠ `updates[].ticketId` is hardcoded to `^ECC-[0-9]{3,}$` even though projects use their own prefixes (FOC-, OTH-, …) — a live schema bug.

---

7. Correct workflows (recipes)

7.1 Creating a task the right way

Today there are exactly two validated creation paths: **`route start --project <ID> --task-title <T>`** (managed launch) and **the ingestion pipeline** (schema-validated proposals). Everything else is a hand-edit of `dashboard.json` .

Every task description/summary must answer **what, why, and how** — 1–2 sentences each, no leading commit hashes, branch names, or internal jargon. A complete task also carries: `projectID` , `priority` , `category` , `summary` , `next` , `doneWhen` , and — once worked — `worktreePath` , `branch` , `plan.md` , and at least one well-formed subtask.

If you must write `dashboard.json` directly (you shouldn't — see §8): take `.<name>.focus-queue.lock`, preserve unknown keys and `queue-order`, write atomically, keep types exact, and never reuse or duplicate an ID across dashboards.

7.2 Registering an agent on a task

1. **Immediately at startup:** `focus-queue-session attach "$FOCUS_QUEUE_TASK_ID"` (or the known ticket ID). Repeat after switching tickets. Never run `tabname` or overwrite labels afterward.
2. If no ticket is known: `focus-queue-session placement --prompt "<initial prompt>"` from the intended repo, then follow the capsule; `identify` once IDs exist.
3. Read `$FOCUS_QUEUE_AGENT_CONTEXT` (placement capsule) before editing source. Stop on `blocked-conflict`.
4. The registration record must name: task, provider session UUID, tmux session (managed panes), and worktree + branch (isolated workspaces). The first recorded workspace is immutable; resuming reuses the exact workspace and provider UUID.
5. Heartbeat with `progress`, log with `log`, finish with `complete <TASK> <SUBTASK> "<summary>"`.

7.3 Router recipe

1. `route inspect --prompt "<complete objective>" --cwd "$PWD" --provider <p>` → packet with candidates and command templates.
2. Exactly one `route start <routeID> --task <ID> --title ... --description ...` (the objective is SHA-256-bound; a paraphrased prompt is rejected, the saved original is used).
3. Exit only when the result says `registered` with `routerAction: exit`; otherwise report the recovery locator and stay available.

---

8. Failure analysis — where this structure breaks

These are the concrete mechanisms behind the "tasks written incorrectly", "session recall fails", and "tmux sessions lost" issues.

8.1 Task creation bypasses validation

The app validates writes made *through* `JSONDashboardRepository` (revision check, duplicate-ID rejection, type-safe re-encode). But **no CLI command creates a ticket**, so agents and the orchestrator hand-edit `dashboard.json`, which gets only *read-time* validation:

- Missing fields silently default (`title` ←id, `priority` ←medium, `status` ←backlog, `summary` ←"") — the task "exists" but is hollow: no what/why/how, no `next`, no `doneWhen`.
- Wrong types kill the entire dashboard load, not just the one ticket.
- `worktreePath` was absent in **19 of 39** audited task contexts (`Docs/DATA_BACKFILL_AUDIT.md`) — and the audit concluded provider/session/path correlation **cannot be reconstructed from data alone** afterward. Data written wrong at creation is permanently unrecoverable.

- The audit tool (`focus_queue_data_audit.py`) already knows the required field set — but nothing *enforces* it at write time.

8.2 `state.json` and `config.json` are unsafe external-write targets

`state.json` edits are overwritten by the app's in-memory cache. `config.json` edits are ignored until restart and then silently lost on the next settings save (only unknown keys survive). Any agent "fixing" registration by editing `state.json` is racing the app and will lose.

8.3 Session recall is intentionally narrow — and the gaps show

Recall verifies the full saved tmux identity (server PID, creation time, session name, pane ID/PID, TTY) plus matching provider UUID. It **never** scans transcripts, runs `codex resume` / `claude --resume` , or starts a replacement. So recall fails whenever:

- the tmux server rebooted (resurrect restores shells, but they never get the live-agent marker back),
- pane IDs were reused or the terminal transport died mid-launch,
- the provider UUID was never captured (short-lived calls that closed before discovery),
- the record was written by hand and is missing `tmux` , `paneID` , or `sessionID` fields.

`runtime locate` is the honest probe here — but there is no `session resume` command that rebinds a *known-good* provider UUID to a *new* pane/worktree. That gap is why recalling sessions "occasionally" fails and no one can fix it from the CLI.

8.4 tmux lifecycle traps

- Reboot/server death ends all agents; restored shells are not agents.
- Duplicate-resume guards compare full server/pane/process/TTY references; an unconfirmed launch blocks retries forever until the startup sweep (30 s) completes-or-fails it.
- `/tmp` vs `/private/tmp` canonicalization and cross-prefix task ownership both broke routing historically — fixed, but they show how identity comparisons are brittle at every seam.

8.5 Validation inconsistencies in the CLI itself

- `task status` accepts any string (priority is validated; status is not).
- `route start` silently drops unknown flags.
- `agents clear --all` destroys all registration records with no confirmation and leaves processes running untracked.
- Kimi cannot `attach` from env (no UUID env var) though the usage text implies it can.
- The proposal schema's `ticketId` is hardcoded to `ECC-` while projects have their own prefixes.

8.6 The orchestrator's authority problem

Per `Docs/ACTION_BOUNDARIES.md` , the queue-chat orchestrator now has real mutation capability, but **ordinary questions share the write-capable path** — there is no typed request authority (`answer` vs `proposeTaskEdit` vs `applyTaskEdit`). Combined with §8.1 (no validated `task create`), the orchestrator can currently only

mutate fields by hand-editing JSON with partial validation. That is precisely the configuration that produces malformed tasks.

---

9. Missing commands — suggested additions

Ordered by impact. Each is designed to close a failure mode from §8.

Tier 1 — close the task-creation hole

1. **`task create --project <ID> --title <T> --summary <S> --next <N> --done-when <D> [--priority p] [--category c] [--description-file path]`**
   The single most important missing command. Allocates the next `<PREFIX>-<n>` ID under the dashboard lock, validates the full field set through `JSONDashboardRepository`, **requires** `summary` (what/why/how), `next`, and `doneWhen` (refuse hollow tasks), creates `tasks/<ID>/` with a scaffolded `context.json` + `plan.md`, and emits the ticket JSON. This makes the correct path the easy path.
2. **`task show <TASK-ID> [--json]`** — read one ticket + its context completeness. Today agents must parse two files to know a task's state.
3. **`task set <TASK-ID> <field> <value>` / `task edit`** — generic field mutation with per-field validation (enums for status/priority/category, ISO dates, URL shapes for links), so the orchestrator can "change fields" without hand-editing JSON. Fix `task status` to validate against the known vocabulary (or document the app's free-form statuses as an enum).
4. **`task validate <TASK-ID> [--repair]`** — check a ticket + context against the same field list `focus_queue_data_audit.py` uses (`title, summary, successCriteria, worktreePath, planPath, subtasks…`); `--repair` fills only safe defaults and reports what it could not infer. Never invent content — report, don't fabricate.

Tier 2 — session recall and agent inventory

1. **`sessions list [--task <ID>] [--provider p]`** — dump `agentSessions[]` + subtask correlation. Currently there is no read path for the registry.
2. **`session resume <TASK-ID> <SUBTASK-ID> [--new-pane]`** — the recall command: verify provider UUID + workspace via `runtime locate`, rebind to a fresh tmux pane, update the session record, emit the resume command it used. Encodes the REMOTE_SESSION_RECALL rules in one place instead of relying on every agent to remember them.
3. **`doctor [--task <ID>]`** — whole-system health: orphaned sessions (recorded but `runtime locate` = stopped), subtasks with sessions but no workspace, tasks with agents but no `worktreePath`, stale startup receipts, wedged router slots. Read-only; exit code = number of problem classes found.
4. **`attach` should verify the ticket exists** on some dashboard and warn (or refuse without `--force`) when it doesn't.

Tier 3 — robustness & guardrails

1. **Reject unknown flags** in `route start` (and audit every hand-rolled parser) — silent drops turn typos into misprovisioned agents.
2. **`agents clear --all` needs `--yes` + a pre-flight count**, and should optionally terminate or detach the running processes it untracks.

3. **Kimi session identity**: add a `KIMI_SESSION_ID` env binding (or document hooks as the only path and fix the `attach` usage text).
4. **Generalize the proposal schema `ticketId` pattern** to `^[A-Z][A-Z0-9]{1,4}-[0-9]{3,}$` (matching `ProjectCatalog` 's prefix rule).
5. **`config reload`** command that posts the catalog-change notification (or a new config-change notification) so external config edits become safe-ish.
6. **`--label` should either work or be removed** from the harness CLIs; Kimi's `--effort` / `--disable-tools` should error instead of silently no-op'ing.

---

10. Hardening recommendations

Make the CLI the only write path, then make the CLI strict.

1. **Schema-enforce at write, not just read.** Every mutation that can produce a task/subtask/session record should go through the same validated repository path the app uses. The current asymmetry (strict app writes, lenient hand writes, repair-on-read) guarantees drift.
2. **Refuse hollow tasks.** `summary` (what/why/how), `next` , and `doneWhen` should be required at creation. A task without them is debt that the backfill audit proved cannot be repaid from data alone.
3. **Receipts for everything.** `route start` already writes startup receipts and detects replay. Extend the same pattern to `task create` , `attach` , and `session resume` : idempotent, UUID-keyed, replayable. Idempotency is what makes orchestrator retries safe.
4. **Correlation at creation, not reconstruction.** Every subtask record should be born with `sessionID` , `tmux` reference, and `workspace` (path+branch+baseCommit) filled by the launch path — the launch already knows all three. Registration completeness is already *checked* (`registrationIssue` on handoffs); it should be *blocking* for managed launches.
5. **Typed authority for the orchestrator.** Implement the `ACTION_BOUNDARIES.md` proposal: read-only answers must not share the write path. Mutations should be proposed, validated, then applied — with the proposal itself schema-checked.
6. **One lock discipline, enforced.** Provide a tiny `focus-queue-session exec-lock <path> -- cmd...` (or document the lock paths as part of the contract) so any external writer can't bypass `flock` accidentally.
7. **Router hardening.** The router's one-use, prompt-hash-bound capability is good design. Remaining gaps: max-4 router slots can wedge on terminal-close failure (surfaced, but manual); the app never auto-retries an unresolved route — a `route retry <routeID>` /abandon command would let the orchestrator recover deliberately instead of waiting for the 30 s sweep. Also consider letting `route inspect` results expire, so stale packets can't be claimed against a changed queue.
8. **Ship `doctor` in CI of the data**: run the audit + doctor on a schedule; a queue that knows it's degraded beats one that silently is.

---

11. Quick reference card

```
# Registration (mandatory, first thing)
focus-queue-session attach <TASK-ID>
focus-queue-session identify <TASK> <SUBTASK> "<title>" "<what/why/how>"

# Lifecycle
focus-queue-session progress  <TASK> <SUBTASK> working "summary"
focus-queue-session log       <TASK> <AGENT>   "entry"
focus-queue-session attention <TASK> <AGENT>   question "summary"
focus-queue-session complete  <TASK> <SUBTASK> "result summary"

# Routing (router/orchestrator)
focus-queue-session route inspect --prompt "<objective>" --cwd "$PWD" --provider codex
focus-queue-session route start <ROUTE-UUID> --task <TASK-ID> --title T --description D

# Task fields (validated)
focus-queue-session task rename   <TASK-ID> <title...>
focus-queue-session task priority <TASK-ID> critical|high|medium|low
focus-queue-session task status   <TASK-ID> <status>     # ⚠ unvalidated

# Diagnostics
focus-queue-session context <TASK> | plan <TASK> | diagnose <TASK>
focus-queue-session runtime locate <provider> <session-uuid>
focus-queue-session reconcile <TASK>

# Never do
#  ✗ hand-edit state.json            (app cache overwrites you)
#  ✗ hand-edit config.json           (ignored until restart, then lost)
#  ✗ write dashboards without the lock and full schema
#  ✗ create tasks without summary/next/doneWhen
```

Generated 2026-10-09 from repository sources. Command grammar is authoritative at `Sources/FocusQueueSession/main.swift:16`; when this document and the usage string disagree, the usage string wins.